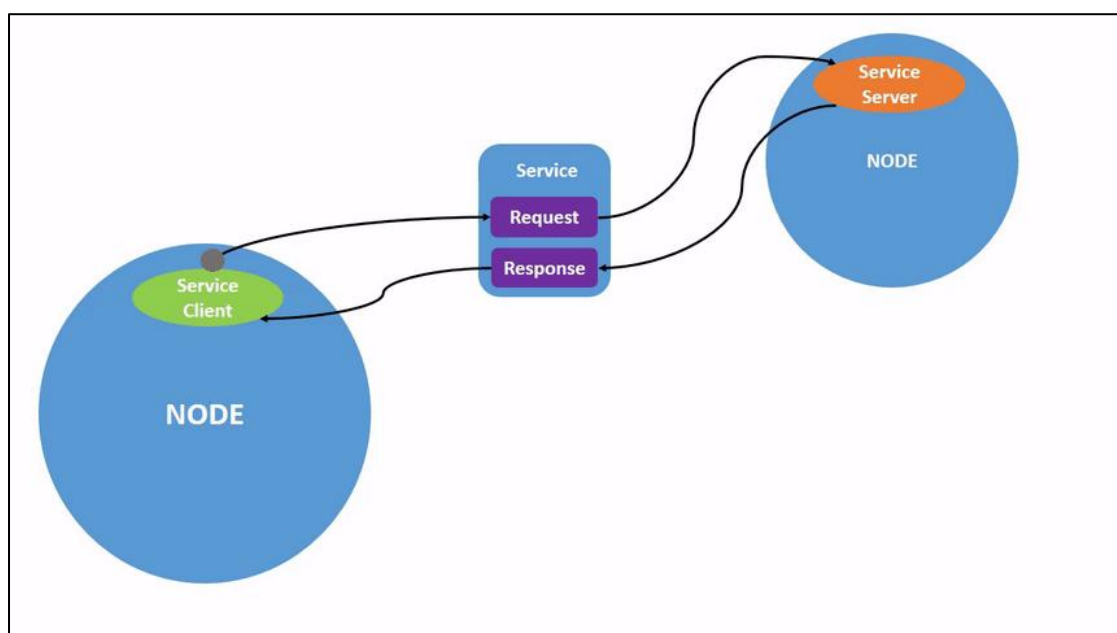


Lesson 9 ROS2 Service Instruction

1. Introduction to Service Communication

Service communication is a communication model based on request and respond. During the mutual communication, the client sends request data to the server, and the server responds with results to the client.

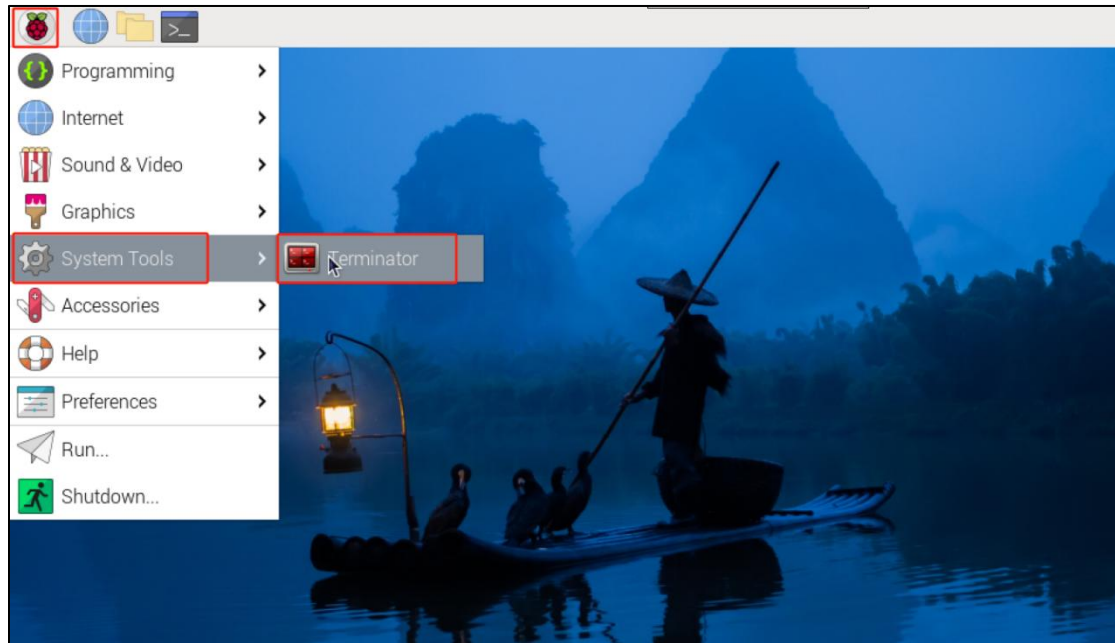


From the perspective of service implementation, the request- respond form is called the client/server model, abbreviated as CS mode. In this model, when the client requires certain data, aiming at a specified service, a request message targeting a specific service will be sent. Upon receiving the request, the server processes it and provides a respond.

This communication mechanism is common in our daily life. For example, when we browse various web pages, your computer browser acts as the client, it sends requests to the website using domain names or other operations. The server returns the necessary page data to be displayed upon receiving these requests.

2. Create Interface

- 1) Click  and select "System Tools -> Terminator" in sequence.



- 2) Enter the command "**cd hiwonder_ws/src/**" to switch to the **demo_interfaces** package.

```
ubuntu@raspberrypi:~$ cd hiwonder_ws/src/demo_interfaces/  
ubuntu@raspberrypi:~/hiwonder_ws/src/demo_interfaces$
```

- 3) Enter the command "**demo_interfaces**" to create the srv folder.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/demo_interfaces$ mkdir srv
```

- 4) Enter the command "**cd srv**" to enter the srv folder.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/demo_interfaces$ cd srv  
ubuntu@raspberrypi:~/hiwonder_ws/src/demo_interfaces/srv$
```

- 5) Enter the command "**vim AddInts.srv**" to edit the program. Enter the code below. If you need to make modifications, press "i" to enter insert mode. Once you've finished editing, press "**Esc**", then type "**:wq**" to save and exit.

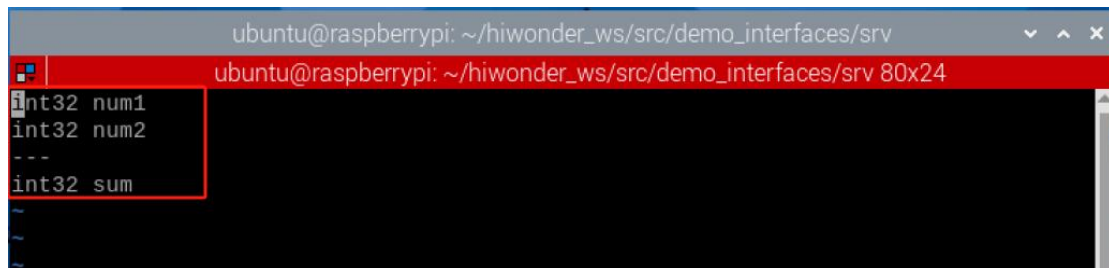
```
ubuntu@raspberrypi:~/hiwonder_ws/src/demo_interfaces/msg$ vim String.msg
```

```
int32 num1

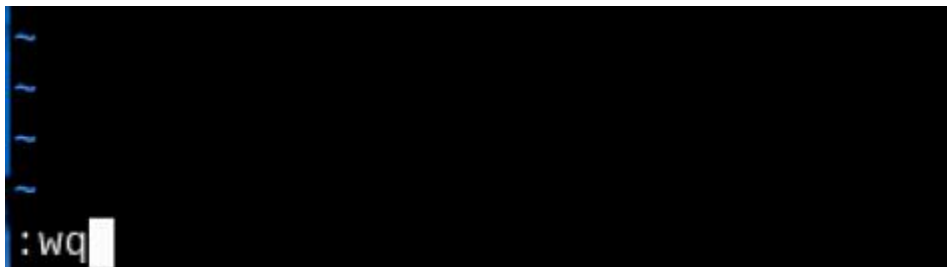
int32 num2

---

int32 sum
```



```
ubuntu@raspberrypi: ~/hiwonder_ws/src/demo_interfaces/srv
ubuntu@raspberrypi: ~/hiwonder_ws/src/demo_interfaces/srv 80x24
int32 num1
int32 num2
---
int32 sum
```



- 6) Enter "**cd ..**" to navigate back to the parent directory, then enter "**vim CMakeLists.txt**" to open the file using the Vim text editor. Copy the following program and paste it into the specific location as pictured. If you need to make modifications, press "i" to enter insert mode. Once you've finished editing, press "**Esc**", then type "**:wq**" to save and exit.

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces( ${PROJECT_NAME}

    "msg/String.msg"

    "srv/AddInts.srv"

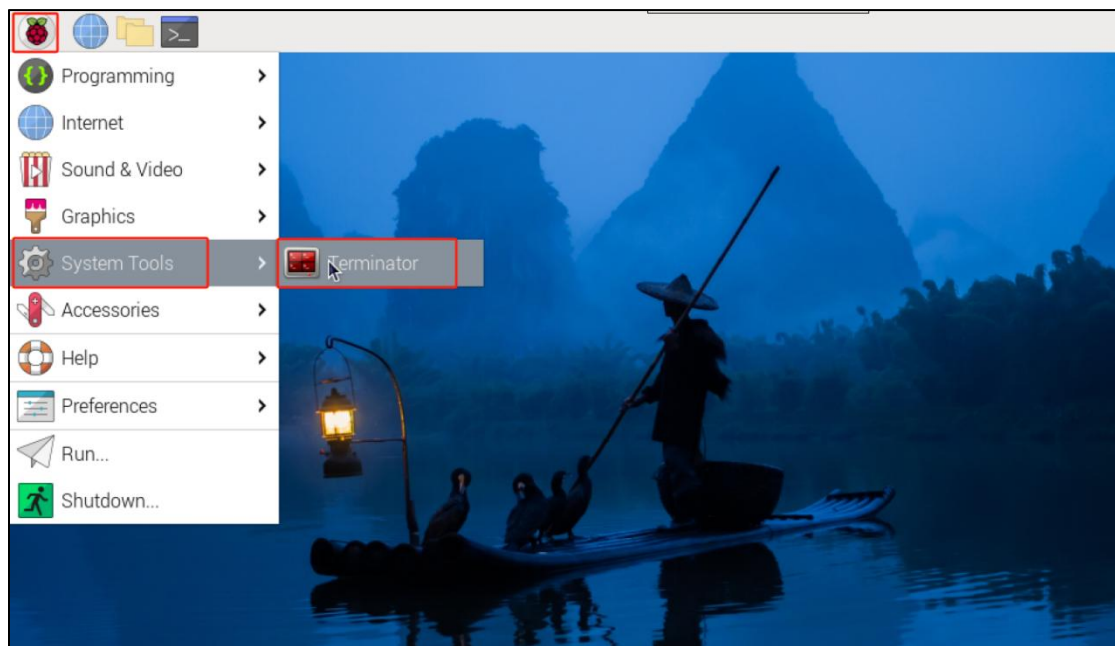
)
```

```
find_package(rosidl_default_generators REQUIRED)
roslint_generate_interfaces( ${PROJECT_NAME}
    "msg/String.msg"
    "srv/AddInts.srv"
)
if(BUILD_TESTING)
    find_package(ament_lint_auto REQUIRED)
```

3. Create Service

3.1 Create Server

- 1) Click  and select "System Tools -> Terminator" in sequence.



- 2) Enter the command "**cd hiwonder_ws/src/**" to switch to the src folder within hiwonder_ws workspace.

```
ubuntu@raspberrypi:~$ cd hiwonder_ws/src/
ubuntu@raspberrypi:~/hiwonder_ws/src$
```

- 3) Enter the command "**ros2 pkg create service_demo --build-type ament_python --dependencies rclpy**" and press Enter to create a package

named “**service_demo**” with rclpy dependency.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ ros2 pkg create service_demo --build-type ament_python --dependencies rclpy
```

4) Enter the command “**cd service_demo/service_demo/**” to switch to the “**service_demo**” package.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ cd service_demo/service_demo/  
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo/service_demo$
```

5) Enter the command “**vim service_server.py**” to edit the program, copy the program below. If modification is required, you can press “i” to modify the program. After modification, press “Esc” and enter “**:wq**” to save and exit.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo/service_demo$ vim service_server.py
```

```
import rclpy  
  
from rclpy.node import Node  
  
from demo_interfaces.srv import AddInts  
  
# Define a MinimalService class inhering from the Node.  
  
class MinimalService(Node):  
  
    # The initialization method of the Class  
  
    def __init__(self):  
  
        # Call the initialize method of the Node class to set the node name as  
        "minimal_service"  
  
        super().__init__('minimal_service')  
  
        # Create a service that provides a service of type AddInts. Set the
```

service name as 'add_two_ints', and the callback function as 'add_two_ints_callback'.

```
self.srv = self.create_service(AddInts, 'add_two_ints',
self.add_two_ints_callback)
```

```
# Define service callback function
```

```
def add_two_ints_callback(self, request, response):
```

```
    # Log the received requests num1 and num2 in the logs.
```

```
    self.get_logger().info('Incoming request\nnum1: %d num2: %d' %
(request.num1, request.num2))
```

```
    # Calculate and set the respond field 'sum'.
```

```
    response.sum = request.num1 + request.num2
```

```
    # Return respond
```

```
    return response
```

```
# Main function
```

```
def main():
```

```
    # Initialize ROS2 node
```

```
    rclpy.init()
```

```
    # Create MinimalService object
```

```
minimal_service = MinimalService()

# Enter the event loop of ROS2 node

rclpy.spin(minimal_service)

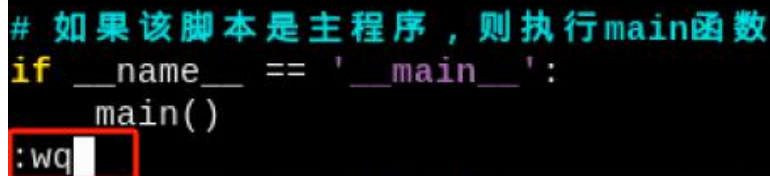
# Shut down ROS2 node

rclpy.shutdown()

# If this script is the main program, main function is executed.

if __name__ == '__main__':

    main()
```



```
# 如果该脚本是主程序，则执行main函数
if __name__ == '__main__':
    main()
:wq
```

6) Enter the command “**chmod +x service_server.py**” and press Enter to grant the executable permission to the saved **service_server.py**.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo/service_demo$ chmod +x service_server.py
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo/service_demo$
```

3.2 Create Client

1) Enter the command “**vim service_client.py**” to edit the program. Copy the following program. If you need to make modifications, you can press “i” to modify the program. After modification, press “Esc” and enter “:wq” to save and exit.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo/service_demo$ vim service_client.py
```

```
import sys

import rclpy

from rclpy.node import Node

from rclpy.logging import get_logger

from demo_interfaces.srv import AddInts


# Define a MinimalClient class inhering from the Node.


class MinimalClient(Node):

    # The initialization method of the Class

    def __init__(self):

        # Call the initialization method of the Node class. Set the node name
        as "minimal_client"

        super().__init__('minimal_client')

        # Create a server and connect to the AddInts service named as
        "add_two_ints".

        self.cli = self.create_client(AddInts, 'add_two_ints')

        # Wait for the service connection, with a maximum wait time of 1
        second.
```



```
while not self.cli.wait_for_service(timeout_sec=1.0):

    self.get_logger().info('service is connecting...')

# Send service request method

def send_request(self):

    # Create an AddInts request object.

    request = AddInts.Request()

    # Get two integers from the command-line arguments and set them
into the request object.

    request.num1 = int(sys.argv[1])

    request.num2 = int(sys.argv[2])

    # Asynchronously call the service and retrieve a Future object.

    self.future = self.cli.call_async(request)

# Main function

def main():

    # Check if the command-line arguments contain two integers.

    if len(sys.argv) != 3:

        get_logger("rcldpy").error("Please provide two integers")

    return
```

```
# Initialize ROS2 node

rclpy.init()


# Create MinimalClient object

minimal_client = MinimalClient()


# Send service request

minimal_client.send_request()


# Block and wait for the service call to complete.

rclpy.spin_until_future_complete(minimal_client, minimal_client.future)


try:

    # Retrieve the respond from the service call.

    response = minimal_client.future.result()


    # Print the respond results

    minimal_client.get_logger().info("Request result: sum = %d" %
response.sum)

except Exception:

    # Print error message if the request fails

    minimal_client.get_logger().error("Request Fail")
```

```
# Destroy node object

minimal_client.destroy_node()


# Shut down ROS2 node

rclpy.shutdown()


# If this script is the main program, the main function is executed.

if __name__ == '__main__':

    main()
```

```
# 如果该脚本是主程序，则执行main函数
if __name__ == '__main__':
    main()
:wq
```

2) Enter the command “**chmod +x service_client.py**” and press Enter to grant the executable permission to the saved **service_client.py** file.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo/service_demo$ chmod +x service_client.py
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo/service_demo$
```

4. setup.py File Settings

The setup.py file defines the metadata and build configuration for a ROS2 package, providing information such as package metadata, dependencies, build configuration, and installation logic. It helps developers correctly build, install, and use ROS2 packages. It is necessary to write the program entry points for service_client.py and service_server.py into the setup.py file.

- 1) Enter the command **cd ..** to switch to the parent directory.

```
ubuntu@raspberrypi1:~/hiwonder_ws/src/service_demo/service_demo$ cd ..
ubuntu@raspberrypi1:~/hiwonder_ws/src/service_demo$
```

- 2) Enter the command “**vim setup.py**” and press Enter to open the setup.py file.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo$ vim setup.py
```

- 3) Press “i” to enter the editing mode, and then enter the following code to the corresponding position.

```
'service_server = service_demo.service_server:main',
'service_client = service_demo.service_client:main'
```

```
tests_require=[ 'pytest' ],
entry_points={
    'console_scripts': [
        'service_server = service_demo.service_server:main',
        'service_client = service_demo.service_client:main'
    ],
},
```

- 4) Enter “:wq” to save and exit the file.

```
'console_scripts': [
    'service_server = service_demo.service_server:main',
    'service_client = service_demo.service_client:main'
],
},
```

5. Compilation and Execution

1) After granting the executing permission, enter the command “**cd ~/hiwonder_ws/**” to switch to the directory of the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo$ cd ~/hiwonder_ws/  
ubuntu@raspberrypi:~/hiwonder_ws$
```

2) Enter the command “**colcon build**” and press Enter to compile the packages within the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws$ colcon build
```

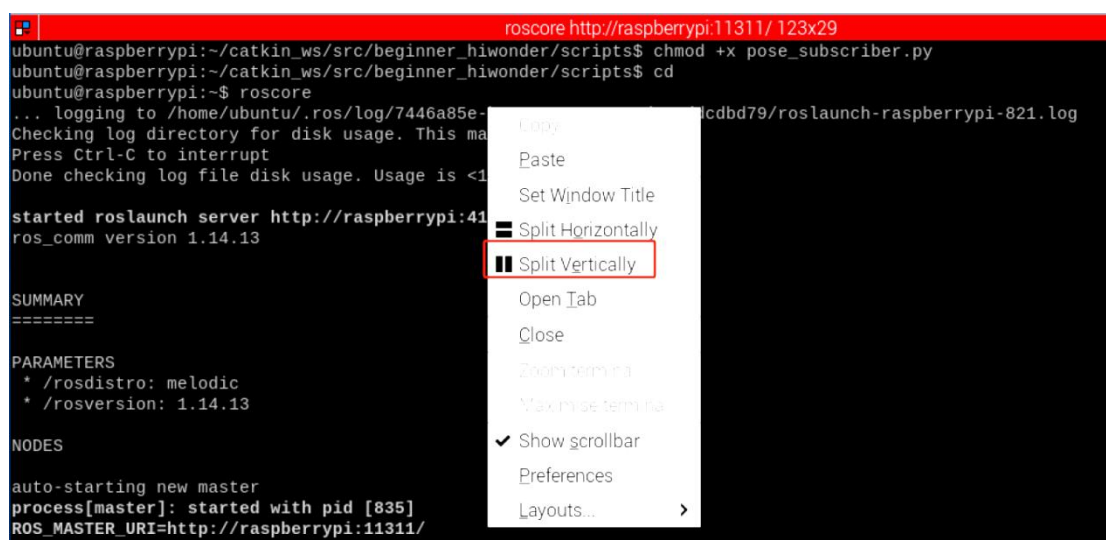
3) Enter the command “**source ./install/setup.bash**” and press Enter to make the environment variables take effect.

```
ubuntu@raspberrypi:~/hiwonder_ws$ source /opt/ros/humble/setup.bash  
ubuntu@raspberrypi:~/hiwonder_ws$
```

4) Enter the command “**ros2 run service_demo service_server**” and press Enter to start service_server server.

```
ubuntu@raspberrypi:~/hiwonder_ws$ source ./install/setup.bash
```

5) Right click to select “**Split Vertically**” to create a new terminal window.



6) Enter the command “**ros2 run service_demo service_client 1 2**” and press Enter to start the **service_client** client to send the calculation request for

numbers 1 and 2. Once the server receives the calculation request for numbers 1 and 2, it will send back the result to the client.

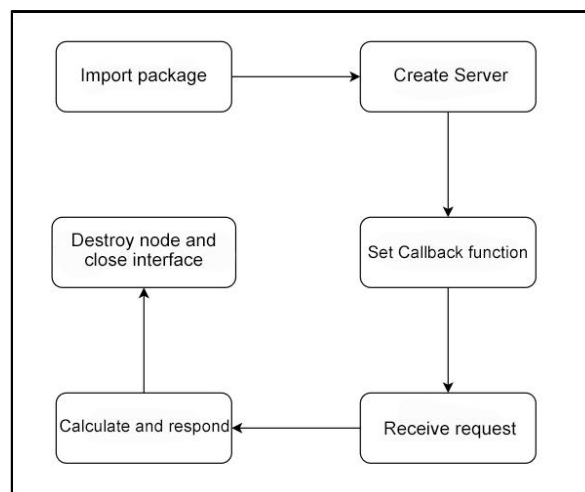
```
ubuntu@raspberrypi: ~ 40x31
ubuntu@raspberrypi1:~$ ros2 run service_demo service_client 1 2
[INFO] [1706239415.291166584] [minimal_client]: 请求结果 : sum = 3
ubuntu@raspberrypi:~$
```

```
ubuntu@raspberrypi:~$ ros2 run service_demo service_server
[INFO] [1706239376.867932340] [minimal_service]: Incoming request
num1: 1 num2: 2
```

6. Program Analysis

6.1 Server

According to the realization result, the logic progress for the program is shown as pictured:



Create a server named "**minimal_service**" which will receive requests, perform calculations on them, and respond with the result. During program execution, ROS 2 node is initialized first. Then, a MinimalPublisher object is created, and the program enters the ROS 2 node's event loop. When the program is interrupted, the node object is destroyed, and the ROS 2 node is shut down.

◆ Main Function

```
# 主函数
def main():
    # 初始化ROS 2节点
    rclpy.init()

    # 创建MinimalService对象
    minimal_service = MinimalService()

    # 进入ROS 2节点的事件循环
    rclpy.spin(minimal_service)

    # 关闭ROS 2节点
    rclpy.shutdown()
```

First, invoke the **rclpy.init()** function to initialize ROS2 Python interface. Then instantiate the **MinimalService()**. Finally, execute the **minimal_service** within the event loop of the ROS2 node.

◆ MinimalService Class

```
# 定义一个继承自Node的MinimalService类
class MinimalService(Node):

    # 类的初始化方法
    def __init__(self):
        # 调用Node类的初始化方法，设置节点名称为'minimal_service'
        super().__init__('minimal_service')

        # 创建一个服务，提供AddInts类型的服务，服务名称为'add_two_ints'，回调函数为add_two_ints_callback
        self.srv = self.create_service(AddInts, 'add_two_ints', self.add_two_ints_callback)

    # 定义服务回调函数
    def add_two_ints_callback(self, request, response):
        # 在日志中记录接收到的请求的num1和num2
        self.get_logger().info('Incoming request\nnum1: %d num2: %d' % (request.num1, request.num2))

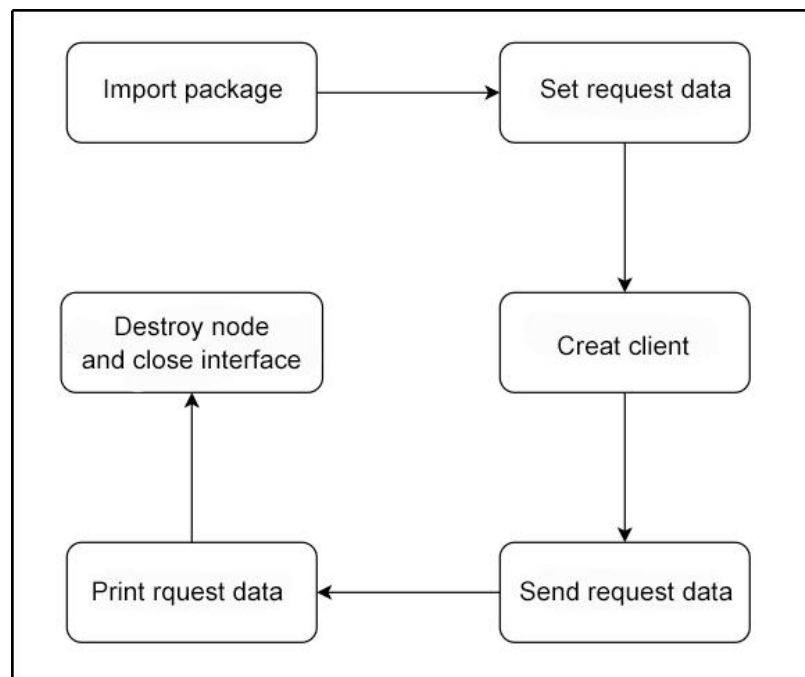
        # 计算并设置响应的sum字段
        response.sum = request.num1 + request.num2

        # 返回响应
        return response
```

First, a node named `minimal_service` is created. Then, a service is created to provide a service of type `AddInts`, with the service name `'add_two_ints'`. Inside the `add_two_ints_callback()` callback function, the two data from the request are received, processed for calculation, and the result of the calculation is responded back.

6.2 Client

According to the realization result, the logic progress for the program is shown as pictured:



A client named **"minimal_client"** is created, which connects to the `AddInts` service named `'add_two_ints'`. It sends the two requested data. During program execution, it initializes the ROS2 node, creates a `MinimalClient` object, sends the service request, prints the response result, and upon interruption, it destroys the node object and closes the ROS 2 node.

◆ Main Function

```
def main():
    # 检查命令行参数是否包含两个整数
    if len(sys.argv) != 3:
        get_logger("rclpy").error("请提供两个整数值")
        return

    # 初始化ROS 2节点
    rclpy.init()

    # 创建MinimalClient对象
    minimal_client = MinimalClient()

    # 发送服务请求
    minimal_client.send_request()

    # 阻塞等待服务调用完成
    rclpy.spin_until_future_complete(minimal_client, minimal_client.future)

    try:
        # 获取服务调用的响应
        response = minimal_client.future.result()

        # 打印响应结果
        minimal_client.get_logger().info("请求结果 : sum = %d" % response.sum)
    except Exception:
        # 打印请求失败的错误信息
        minimal_client.get_logger().error("请求失败")

    # 销毁节点对象
    minimal_client.destroy_node()

    # 关闭ROS 2节点
    rclpy.shutdown()
```

First, invoke the `rclpy.init()` function to initialize ROS2 Python interface. Then instantiate the `MinimalClient()` file and send the service request. Finally, process the respond data.

◆ MinimalClient Class

```
# 定义一个继承自Node的MinimalClient类
class MinimalClient(Node):

    # 类的初始化方法
    def __init__(self):
        # 调用Node类的初始化方法，设置节点名称为'minimal_client'
        super().__init__('minimal_client')

        # 创建一个服务客户端，连接到名为'add_two_ints'的AddInts服务
        self.cli = self.create_client(AddInts, 'add_two_ints')

        # 等待服务连接，最多等待1秒
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('service is connecting...')

    # 发送服务请求的方法
    def send_request(self):
        # 创建一个AddInts请求对象
        request = AddInts.Request()

        # 从命令行参数中获取两个整数，并设置到请求对象中
        request.num1 = int(sys.argv[1])
        request.num2 = int(sys.argv[2])

        # 异步调用服务，并获取一个Future对象
        self.future = self.cli.call_async(request)
```

First, a node named “minimal_client” is created. Then, a service client is created to connect to the AddInts service named “add_two_ints”. In the send_request() function, the two command-line arguments are used as the data for the request object, and the request is sent.